

scripts/pipeline-build-test-distribute.sh — User Guide

Last verified: 2026-08-21 (phase wiring landed — T038; verified by real runs against disposable fixture repositories, see "Verification record" below) **Inheritance:** HelixConstitution §11.4.18 (script documentation mandate) + Lava's Anti-Bluff Pact (Sixth/Seventh Laws) + §6.T.2 (Resource Limits)

Overview

The top-level orchestrator for this project's local build-test-distribute pipeline (specs/002-build-test-distribute-pipeline/). It sequences the pipeline's phase scripts against one shared run identifier, and consolidates their results into a single Pipeline Run Report per invocation.

What is wired today — and what deliberately is not

This section is the authoritative statement of what this script can currently do. It is deliberately explicit, because a pipeline that quietly does less than its name implies is the exact failure this feature exists to prevent.

Phase	Script	Wired?
precondition	scripts/pipeline/phase-00-precondition.sh	☑
build	scripts/pipeline/phase-01-build.sh	☑
test	scripts/pipeline/phase-02-test.sh	☑
install_boot	scripts/pipeline/phase-03-install-boot.sh	✗
live_verify	phase-04-live-verify-api.sh and phase-04-live-verify-api-app.sh	both halves
changelog / distribute	phase-05a-* / phase-05-*	blocked on T040/T041

x

Phase	Script	Wired?
docs refresh	phase-06-*	blocked on the same reviewed change
closure	phase-07-*	blocked on T048/T049 + T054

The distribute, docs and closure phases are **not** unimplemented-by-accident. They are gated behind constitutional amendment tasks that have not been reviewed or merged: T040/T041 (an alternate path under CLAUDE.md §6.AA's Two-Stage Distribute Mandate and the Seventh Law's clause 3) and T048/T049 (a carve-out in the Decoupled Reusable Architecture rule's "submodule fetch/pull is an EXPLICIT operator action, never automatic"). Until those land, this script **cannot** distribute or push anything, by construction.

live_verify covers both surfaces (since T037 landed, 2026-08-21): the running lava-api-go service, and the :api-app debug APK installed onto a Containers-submodule emulator per §6.AH driving its real boot-and-serve Challenge. A phase may own more than one script; they run in listed order and the phase halts at the first failure, so a later script can never append a PASS entry for a phase whose earlier half already failed. Both append their own live_verify entry to phases[] — the report schema permits that (phases[] declares no uniqueItems), and it is the honest shape, since finalize_run_report requires *every* entry to be PASS.

The api-app half establishes §6.AH compliance as a **checked fact**, not a logged claim: a poller samples podman ps --filter name=lava-emu throughout the run, and its provenance record is FAIL if no running emulator container was ever observed. It also requires Gradle's own JUnit XML and the Containers attestation row to agree, treating disagreement as a failure in itself.

Remaining honest caveat: the api-app half runs one AVD (API 34 phone) — the only emulator image cached on this host. That is live-verification, not the §6.AE.2 release-gate matrix.

Usage

./scripts/pipeline-build-test-distribute.sh [options] [repo-path]

Option	Meaning
--until <phase>	Stop after <phase> completes successfully. Default live_verify (the furthest wired phase). This is what makes quickstart.md's per-user-story slices runnable: --until test

Option	Meaning
	is the US1 slice, --until live_verify is US1+US2.
--skip <phase>[,<phase>...]	Do not run the named phase(s). precondition may not be skipped.
-h, --help	Print the usage block and exit 0.

Naming a phase that is not wired (e.g. --until distribute) is a **usage error (exit 2)**, not a silent no-op.

precondition cannot be skipped because it is the FR-000 safety boundary — a pipeline that can be talked out of checking its own preconditions has no safety boundary at all.

The optional positional repo-path is forwarded to every phase script as its own [repo-path] argument. It controls which repository the phases *inspect*; it does **not** relocate the Pipeline Run Report, which is always written relative to the current working directory (per scripts/pipeline/lib/run-report.sh's documented contract). Run this script from the repo root, as with scripts/ci.sh and scripts/tag.sh.

Preconditions (FR-000)

Refuses to proceed (exit 2) unless the working tree is on the master branch with a completely clean git status. This is the pipeline's sole safety boundary against an accidental unattended run from unreviewed or uncommitted work.

Required .gitignore entry. .lava-ci-evidence/pipeline-runs/ MUST be gitignored (it is — .gitignore line 59, task T003). The run report is initialized *before* the precondition check, so that a refusal to start is itself recorded as a run outcome. That means the run's own evidence directory exists on disk by the time FR-000's clean-tree rule is evaluated. Without the ignore rule the pipeline dirties the very tree it is about to test and can then never start. This script detects that specific situation and prints an explicit DIAGNOSIS naming the offending paths rather than leaving you to hunt for it.

Evidence and the run report

Every invocation creates a fresh, timestamped directory at .lava-ci-evidence/pipeline-runs/<UTC-run-id>/ containing a report.json (schema: specs/002-build-test-distribute-pipeline/contracts/pipeline-run-report.schema.json), one phase-NN/ subdirectory per

phase, and the phase's own Evidence Records. Per `research.md` R-010, a later invocation never reads a prior run's directory as input — every run restarts fully from scratch (FR-018).

On **every** exit path once the report exists — success, phase failure, or interrupt — this script runs `recompute_evidence_summary` and then `finalize_run_report`.

The `recompute` step is not bookkeeping; it is load-bearing. `data-model.md`'s Validation rule makes outcome: "PASS" conditional on `evidence_summary.rejected_by_anti_bluff == 0`, and `finalize_run_report` implements that rule literally — but the counter it reads is seeded to 0 by `init_run_report` and is only ever populated by `recompute_evidence_summary` scanning the real Evidence Records on disk. Skipping the `recompute` silently turns the anti-bluff half of the outcome rule into a no-op. Regression coverage: `tests/pipeline/test_run_report_evidence_summary.sh` CASE 3.

Consequently, a run whose phases all exited 0 but whose finalized outcome is FAIL (because an Evidence Record was REJECTED by anti-bluff validation) **exits non-zero**. The finalized outcome is authoritative over the individual phase exit codes.

Exit codes

Code	Meaning
0	Every phase that ran reported PASS, and the finalized outcome is PASS.
1	A phase genuinely failed, or the finalized outcome is FAIL (including the all-phases-passed-but-evidence-rejected case).
2	Usage error, or the FR-000 precondition check failed (propagated verbatim from <code>phase-00-precondition.sh</code> , never hardcoded).

Individual phase scripts

Per FR-005, every phase is independently invocable outside this orchestrator. All phase scripts except `phase-00` share the signature `<run_id> [repo-path]` and append their own `phases[]` entry to the shared `report.json`; `phase-00-precondition.sh` takes only `[repo-path]` and this orchestrator appends on its behalf. See `specs/002-build-test-distribute-pipeline/contracts/cli-contract.md`.

Verification record

The phase wiring was verified by real execution against disposable fixture git repositories (never against real master):

- Clean master fixture, --until precondition → exit 0, outcome: PASS, exactly one phases[] entry.
- Fixture dirtied with a real tracked-file change → exit 2, outcome: FAIL, halted at precondition, no false-positive diagnosis.
- Fixture with the .gitignore rule removed → exit 2 **plus** the DIAGNOSIS block naming the untracked run-directory paths.
- All five usage guards (--help, unknown --until, not-yet-wired --until distribute, --skip precondition, unknown option) → exit as specified.

The full end-to-end run across all five wired phases is task **T062**, a review gate that must first be executed on a disposable branch — it has deliberately **not** been run against real master.

Falsifiability

scripts/pipeline/lib/anti-bluff-validate.sh, scripts/pipeline/lib/run-report.sh's evidence aggregator, and scripts/pipeline/phase-00-precondition.sh each carry a hermetic suite under tests/pipeline/, each built RED-then-GREEN. The evidence aggregator additionally carries a recorded mutation rehearsal: disabling its REJECTED detection makes CASE 3 fail with outcome is 'PASS', expected FAIL, and reverting restores the pass.

Maintenance

When this script is modified, update this document in the same commit (CM-SCRIPT-DOCS-SYNC requires it). Per §11.4.18, the documentation **MUST** stay in sync with the codebase.

Cross-references

- specs/002-build-test-distribute-pipeline/spec.md, plan.md, research.md, data-model.md, contracts/, tasks.md — the full design and task breakdown this script implements.
- docs/helix-constitution-gates.md — gate inventory.
- HelixConstitution Constitution.md §11.4.18 (the mandate).